Homework 5 - Greedy Programming

Course: CIS 387 - Analysis of Algorithms - Fall 2026
Instructor: Dr. Aaron Mininger

Submission Instructions

Due Date: Nov 12 at 11:59pm

Zip up the 2 java files and upload to Canvas. **Your name must be at the top of each java file**

Note: You may not use any AI assistance or direct outside help on this assignment.

For 2 extra credit points, answer the following questions (leave a comment on Canvas):

1. Give an estimate of how much time you spent on this assignment.
2. How difficult did you find this assignment?
3. Was anything unclear, or is there any other feedback you would give?

Your responses will be helpful feedback for refining these assignments in future courses.

Grading Information

Points	Description
20	Effort: Does the program compile and run? Was a reasonable attempt made?
20	Part 1: Correct **`RoomSchedule.java`** program.
10	Part 1: **`maxRoomAssignment`** demonstrates an efficient greedy programming solution.
30	Part 2: Correct **`Trucking.java`** program.
10	Part 2: **`minTrips`** demonstrates an efficient greedy programming solution.

Setup

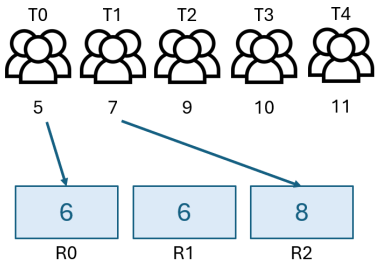
All the code and files needed for this homework are available on the course git repository:
`https://github.com/amininger-mu/CIS387F26Code`

You will see starter files for each part that you should fill in. In addition, there are test files you can use to test each part (you should make additional test files for larger cases!).

1. Conference Room Scheduling

Imagine you have a large company with *T* teams, each team is made up of a certain number of people. You want to assign as many teams to conference rooms as possible. Each conference room has a certain max capacity. What is the most conference rooms that you can fill at the same time (with 1 team per room)?

For example: if you had 5 teams **`T = [ 5, 7, 9, 10, 11 ]`** and 3 conference rooms **`R = [ 6, 6, 8 ]`**, then you could fill at most 2 conference rooms (**`R[0] <- T[0], R[2] <- T[1]`**)



Directions:

You are given a file **`RoomSchedule.java`** which has some starter code to read a test case from stdin.

Implement the method **`List<Assignment> maxRoomAssignment(int[] teams, int[] rooms)`** using an iterative, greedy solution. It should return the optimal room assignments (a list of **`Assignment`** records, where each has a room index and team index).

In the example above: return **`[ {r:0, t:0}, {r:2, t:1} ]`** (room 0 has team 0, room 2 has team 1)

Your solution should not use any recursion. Refer to the code examples in **`lesson22`** (e.g. **`MaxActivities.java`** and **`KnapsackFr.java`**) as well as the slides on greedy programming.

Think: Given the unassigned rooms/teams, what is the greedy choice?

Testing:

The **`RoomSchedule.java`** main function will read in the test case from standard in as:

- An int **`T`**, followed by **`T`** ints (one for each team size)
- An int **`R`**, followed by **`R`** ints (one for each room size)

Example: **`room_test1.txt`**:

```
5
5 7 9 10 11
3
6 6 8
```

You can feed in a test case using:

```
cat room_test1.txt | java RoomSchedule.java
```

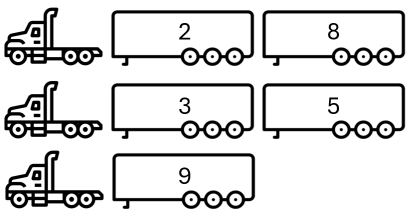
I have provided 1 test case file, but you should make others to test your algorithm.

2. Trucking Transport

Imagine you run a trucking company and you have a number of trailers *T* that you need to transport between warehouses. Your truck can either pull 1 or 2 trailers, but there is a max weight it can pull.

Determine the fewest number of trips you need to tranport all the trailers.

Example: Suppose you have 5 trailers **`[2, 3, 5, 8, 9]`** and a weight limit of 10
Solution: You can do it in 3 trips: **`(2,8) (3,5) (9)`**



Min Trips: 3

Directions:

You are given a file **`Trucking.java`** which has some starter code to read a test case from stdin.

Implement the method **`List<Trip> minTrips(int[] trailers, int limit)`** using an iterative, greedy solution. It should return a minimal list of trips needed to transport all the trailers.

Each **`Trip`** in the list is a **`record Trip(int t1, int t2)`** that contains the indices of the two trailers. If that trip only involves 1 trailer, make **`t2 = -1`**.

In the example above: sol = **`[ (0, 3), (1, 2), (4, -1) ]`**

Your solution should not use any recursion. Refer to the code examples in **`lesson22`** (e.g. **`MaxActivities.java`** and **`KnapsackFr.java`**) as well as the slides on greedy programming. You will be graded on both correctness and efficiency.

Think: Given the available trailers remaining, what is the greedy choice?

Testing:

The **`Trucking.java`** main function will read in the test case from standard in as:

1. 2 ints: the weight limit **`W`** and the number of trailers **`T`**
2. T ints: the weight of each trailer (**`T[i] <= W`**)

Example: **`truck_test1.txt`**:

```
10 5
2 3 5 8 9
```

You can feed in a test case using:

```
cat truck_test1.txt | java Trucking.java
```

I have provided 1 test case file, but you should make others to test your algorithm.